

1.1P: Preparing for OOP – Answer Sheet

Introduction

This paper's answer sheet serves two purposes:

- A. It serves as a revision for you of your previous learnings; and
- B. It establishes a baseline understanding of your knowledge in key Computer Science topics.

As such this paper is divided into the following areas of knowledge:

- A. Your experience with UNIX/DOS console commands;
- B. Your ability to differentiate between data types (e.g. text) and information categories (e.g. Ktle);
- C. Your experience with compiler parsing and evaluation of expressions according to rules of precedence (e.g. BODMAS, also known as GEMS or PEMDAS);
- D. Your understanding of Computer Science concepts and various compiler constructs such as blocks and scope;
- E. Finally taking three steps, we want you to develop a program as follows:
 - 1. starting with a simple function: you provide the pure logic and calculations, no input, nor output;
 - 2. Then, in the second step, you write the main line code that invokes that simple function. Your main line code will provide the necessary data, and then you will print out the result of the function's calculation.
 - 3. Finally we want you to add business logic to the main line program's code; that business logic will interpret the results of the function, and inform your user with information about the results.

Section A: Console commands

- 1. Explain the following terminal instructions:
 - a. cd: Change Directory
 - b. pwd: Print Working Directory
 - c. mkdir: make directory
 - d. cat: concatenate
 - e. ls: list files

Section B: Data types and Information categories

1. Consider the following categories of information, and suggest the most appropriate data type to store and represent each kind of information:

Information Category	Suggested Data Type
A person's family name	String
A person's age in years	Integer
A person's weight in Kilograms	Float
A telephone number	Integer
A temperature on the Kelvin scale	Integer
The average age of a group of children	Float
Whether the student passed this task	Boolean

2. Aside from the examples already provided above, please come up with your own examples of information that could be stored as:

Data Type	Suggested Information Category
String	A name
Integer	A year
Float	The length of something
Boolean	Whether something is red

Section C: Compiler evaluation of expressions

1. Fill out the **last** two columns of the following table based on the expression and values we have supplied.
2. Evaluate the value of each expression under column 1, given its formula, values, and variables; use the given values (column 2) of any variable(s) in the expression.
3. Identify the value of the results (column 3), and the data type the result is most likely to be (column 4) in a compiler "friendly" form (e.g. Float):

Expression	Given	Result	Data Type
------------	-------	--------	-----------

76		76	integer
True		true	Boolean
a	a = 3.1415927	3.1415927	float
1 + 2 * 3 + 4		11	integer
a and False	a = True	false	Boolean
a or False	a = True	true	Boolean
a + b	a = 1 b = 3	4	Integer
3 * a	a = 5	15	Integer
a * 2 + b	a = 2.5 b = 3	8.0	Float
a + 2 * b	a = 2.5 b = 3	8.0	Float
(a + b) * c	a = 2 b = 4 c = 6	35	integer
"Fred" + " Astair"		Fred Astair	String
a + " Rogers"	a = "Ginger"	Ginger Rogers	String

Section D: Compiler Constructs and CS Concepts:

1. Using some code as an example, please explain the difference between **declaring** and **initialising** a variable.

The difference between the two is declaring is creating a variable while initialising is assigning a variable information like `index = 0`, `index` is declaring that there is now a variable called `index` and then the `0` assigns information to the variable

Explain the term **parameter**. Write some **code** that demonstrates a simple use of a parameter. You should show a procedure or function that uses a parameter, and how you would call that procedure or function. A parameter is...<finish the explanation>

In this code “name” is a parameter and the use of parameters are to pass variables between functions, if you define a variable in one function and you want to pass the variable to another function, you must call the function with its parameter

```
def print_silly_name(name):  
    index = 0  
    print(f"{name} is a ", end='')  
    while (index < 60):  
        print("silly name ", end='')  
        index += 1  
    print("name!")
```

Paste your example code below:

2. Using an **coding example**, describe the term **scope** as it is used in procedural programming (not in business nor project management). Make sure you explain the differences of as many kinds of scope that you can identify (at least two, and up to five).

Scope defines availability of a variable, in the code below, “data” and “val” are both have a global scope while “i” has a local scope as it is within a function

```
data = [-2]  
val = 6  
def whatshouldthisfunctionbecalled?(data, val)  
    i = 0  
    result = false  
    while i < data.length  
        if data[i] == val  
            result = true  
            return result  
        end  
        i = i + 1  
    end  
    return result  
end  
result = whatshouldthisfunctionbecalled?(data, val)  
puts result
```

Paste your example code below:

Section E: Implementing Algorithms, Data Handling, and Informing Results - Personalized Requirements

STEP 1:

1. In a procedural style, in any language you prefer, write a function called Average, which accepts an array of integers, and returns the average of those integers.
2. **Do not use any libraries for calculating the average:** we want to see your understanding of algorithms.
3. You must demonstrate appropriate use of parameters, returning and assigning values, and the use of loop(s). **Note — just write the function at this point.** In the next step we will ask you to *invoke the func-on*.
4. You should **not** have a complete program, **nor** even code that outputs anything at this stage. This is a **function**; and input/output and any business logic processing is the responsibility of the (main line) calling code.

```
def average(data):  
    count = len(data)  
    total = sum(data)  
    total = total / count  
    total =str(total)  
    print("your average is: " + total)
```

Paste your example func.on code below:

STEP 2:

5. Using the same preferred language, write the main line calling code you would need to (a) marshal the data, (b) invoke the function, (c) print out the result, and (d) **print out your student name and student Id**
6. We do **not** require you to provide any input processing logic; you simply have provide the inline instance of a collection of data values (provided below) for the function to calculate the average of that data set.
 - a. Sample data values
2.5, -1.4, -7.2, -11.7, -13.5, -13.5, -14.9, -15.2, -14.0, -9.7, -2.6, 2.1
7. Note: you should have made **no changes** to your function.

```
data = []

def averge(data, name, id):
    count = len(data)
    total = sum(data)
    total = total / count
    total = str(total)
    print("your avarage is: " + total)
    print("your name is: " + name)
    print("your student id is: " + id)

def main():
    name = input("what is your name?\n")
    id = input("what is your student id?\n")
    count = input("how many numbers do you have?\n")
    index = 0
    count = int(count)
    while index < count:
        index += 1
        num = input("enter a number\n")
        num = float(num)
        data.append(num)
        averge(data, name, id)

main()
```

Paste all of your example code below:

Paste your example code's output here:

```

what is your name?
Callum
what is your student id?
103995219
how many numbers do you have?
12
enter a number
2.5
enter a number
-1.4
enter a number
-7.2
enter a number
-11.7
enter a number
-13.5
enter a number
-13.5
enter a number
-14.9
enter a number
-15.2
enter a number
-14.0
enter a number
-9.7
enter a number
-2.6
enter a number
2.1
your avarage is: -8.258333333333333
your name is: Callum
your student id is: 103995219

```

8. Using the same preferred language, add to your existing main line code above, the following business logic code for interpreting the result of the funcKon's calculators.
9. Print the message "Multiple digits" if the average is above or equal to 10. Otherwise, print the message "Single digits".
10. And then, if the average is negaKve, add an addiKonal line of output staKng "Average value negative".
11. Finally, if the last digit of the average is larger than the last digit of your Student ID, please print the message "**Larger than my last digit**". Otherwise, please print the correct message, either "**Equal to my last digit**" or "**Smaller than my last digit**".

12. Note, you should not have made any changes to your implemented function
13. Provide evidence of your program running, i.e. the code, its environment, and its run Kme outputs.

```
data = []

def averge(data, name, id):
    count = len(data)
    total = sum(data)
    total = total / count
    total = str(total)
    dig = int(id[8])
    tdig = int(total[-1])

    print("your avarage is: " + total)
    print("your name is: " + name)
    print("your student id is: " + id)
    total = float(total)
    if total >= 10:
        print("Multiple digits")
    elif total < 0:
        print("Average value negative")
    else :
        print("Single digit")
    if tdig > dig:
        print("Larger than my last digit")
    elif tdig < dig:
        print("Smaller than my last digit")
    elif tdig == dig:
        print("Equal to my last digit")

def main():
    name = input("what is your name?\n")
    id = input("what is your student id?\n")
    count = input("how many numbers do you have?\n")
    index = 0
    count = int(count)
    while index < count:
        index += 1
        num = input("enter a number\n")
        num = float(num)
        data.append(num)
    averge(data, name, id)

main()
```

Paste your example code's output here:

```
what is your name?  
Callum  
what is your student id?  
103995219  
how many numbers do you have?  
12  
enter a number  
2.5  
enter a number  
-1.4  
enter a number  
-7.2  
enter a number  
-11.7  
enter a number  
-13.5  
enter a number  
-13.5  
enter a number  
-14.9  
enter a number  
-15.2  
enter a number  
-14.0  
enter a number  
-9.7  
enter a number  
-2.6  
enter a number  
2.1  
your avarage is: -8.258333333333333  
your name is: Callum  
your student id is: 103995219  
Average value negative  
Smaller than my last digit
```

Finally on a new page paste a SINGLE screenshot of your program (main line and func.on) running with its outputs here:

```
EXPLORER
...
test
algorithm.py
Code > python > test > ...
1 h = 103995219
2
3
4
5
6
7
8
9
10
11
12
13
14
15
16
17
18
19
20
21
22
23
24
25
26
27
28
29
30
31
32
33
34
35
36
37
38
39
40
41
Code > python > algorithm.py > average
1 data = []
2
3 def average(data, name, id):
4     count = len(data)
5     total = sum(data)
6     total = total / count
7     total = str(total)
8     dig = int(id[8])
9     tdig = int(total[-1])
10
11     print("your avarage is: " + total)
12     print("your name is: " + name)
13     print("your student id is: " + id)
14     total = float(total)
15     if total >= 10:
16         print("Multiple digits")
17     elif total < 0:
18         print("Average value negative")
19     else :
20         print("Single digit")
21     if tdig > dig:
22         print("Larger than my last digit")
23     elif tdig < dig:
24         print("Smaller than my last digit")
25     elif tdig == dig:
26         print("Equal to my last digit")
27
28 def main():
29     name = input("what is your name?\n")
30     id = input("what is your student id?\n")
31     count = input("how many numbers do you have?\n")
32     index = 0
33     count = int(count)
34     while index < count:
35         index += 1
36         num = input("enter a number\n")
37         num = float(num)
38         data.append(num)
39     average(data, name, id)
40
41 main()
```

what is your name?
Callum
what is your student id?
103995219
how many numbers do you have?
12
enter a number
2.5
enter a number
-1.4
enter a number
-7.2
enter a number
-11.7
enter a number
-13.5
enter a number
-13.5
enter a number
-14.9
enter a number
-15.2
enter a number
-14.0
enter a number
-9.7
enter a number
-2.6
enter a number
2.1
your avarage is: -8.258333333333333
your name is: Callum
your student id is: 103995219
Average value negative
Smaller than my last digit
PS C:\uni stuff\comsci stuff\Code>

End of Task

Please render your paper as a PDF and submit via CANVAS.